

Documentation CRUD Notification

Introduction

This document offers an explanation of the code and the different difficulties encountered during our experience

Objectives

The objective of this assessment is to create a CRUD Notification system with NestJS and MongoDB mainly.

There are other sub objectives such as creating a login system and a front end that we will realise.

Tree description

```
crud-notification-nest
├── dist
├── node_modules
├── src
│   ├── auth
│   │   ├── auth.controller.ts
│   │   ├── auth.module.ts
│   │   └── auth.service.ts
│   ├── notification
│   │   ├── schemas
│   │   │   └── notification.schema.ts
│   │   ├── notification.controller.ts
│   │   ├── notification.module.ts
│   │   ├── notification.repository.ts
│   │   └── notification.service.ts
│   ├── users
│   │   ├── users.module.ts
│   │   └── users.service.ts
│   ├── app.controller.spec.ts
│   └── app.controller.ts
```

```
| |— app.module.ts
| |— app.service.ts
| |— main.ts
```

How to run the code

Of course, to run the code, you just need to execute :

npm run start

The server will be launched at this address :

localhost:3000

To access the notification part, you just need to go to **/notifications**.

The different phases of the project

First Phase : Trainings

-

Second Phase : Coding of the CRUD system

1/ Implementation

The objective at first was first to succeed in creating a notification. At the beginning, I had to know what type of notifications I wanted to create. That's why I decided to add a schema depicting the different attributes of a notification. There is a title, a message, and a date of creation (the current date) :

```

@Schema()
export class Notification {
  @Prop({ required: true })
  title: string;

  @Prop({ required: true })
  message: string;

  @Prop({ default: Date.now })
  createdAt: Date;
}

```

After that, we can go to the notification.repository to manage the database. We first implemented a create method to add a notification. It is then called by the notification.service and latter the notification.controller with the decorator @Post that refers to a HTTP request that can modify the database :

```

@Post()
create() {
  const notification = {
    title: "new notification",
    message: "this is a test",
    date: new Date()
  };
  return this.notificationService.create(notification);
}

```

Then, we do the same with a method to delete, update and to list the whole database.

2/ Testing

2/ Challenges faced

Third Phase : Adding sub objectives